

 Описание проекта для программиста

СКУД Тестирование v3.0.2

1. Общее описание

Проект представляет собой серверное приложение для управления оборудованием СКУД (системы контроля и управления доступом) и проведения автоматизированных тестов контроллеров Z-5R Net.

Основные функции:

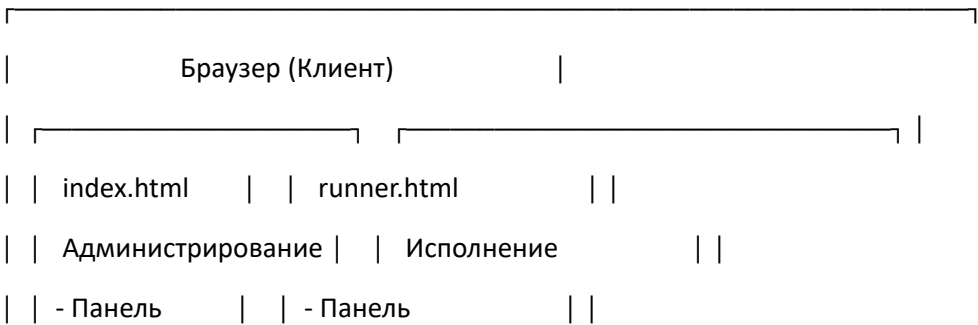
- Обнаружение и управление контроллерами через конвертеры Z397IP
- Создание и редактирование тестовых сценариев (стенды, схемы, тесты, проверки, группы)
- Выполнение тестов с различными режимами (авто, пошагово, с паузами)
- Сохранение результатов и эталонов
- Сравнение результатов с эталонами

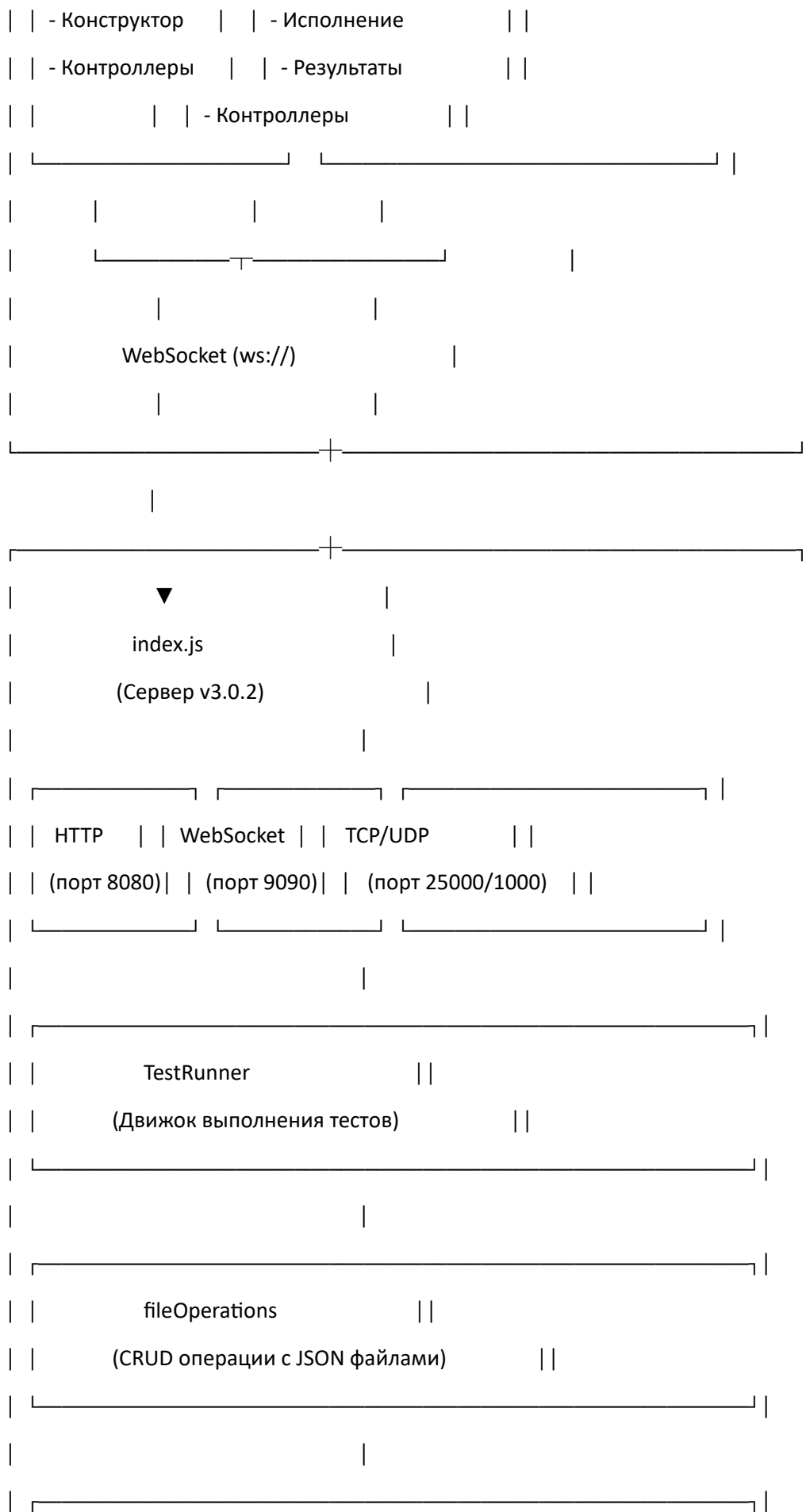
2. Технологический стек

Компонент	Технология	Версия
Сервер	Node.js	12.x+
Протоколы	HTTP, WebSocket, TCP, UDP	-
WebSocket	ws	7.5.12
Работа с ZIP	adm-zip	0.5.9
Клиент	HTML, CSS, JavaScript	ES6

3. Архитектура

text





		Контроллеры и Конвертеры		
		(Управление оборудованием)		



	Файловая система			
	/stands/	- JSON файлы стендов		
	/diagrams/	- JSON файлы схем		
	/diagrams/files/	- PDF файлы схем		
	/tests/	- JSON файлы тестов		
	/checks/	- JSON файлы проверок		
	/groups/	- JSON файлы групп		
	/results/	- JSON файлы результатов		
	/test_sessions/	- JSON файлы сессий тестов		
	/site/	- статические HTML/CSS/JS файлы		

4. Команды API (WebSocket)

Управление контроллерами

Команда	Описание	Параметры
controllers_list	Получить список контроллеров	-
converters_list	Получить список конвертеров	-
available_controllers	Получить активные контроллеры	-
green	Включить зеленый светодиод	controller, interval
red	Включить красный светодиод	controller, interval

Команда	Описание	Параметры
sound	Включить звук	controller, interval
lock	Управление замком	controller, interval
d0/d1	Дискретные выходы	controller, interval
open_for_out/open_for_in	Открыть	controller
new_address	Сменить адрес	controller, new_address
trigger_add	Добавить триггер	controller, type, signal
trigger_dell	Удалить триггер	controller, trigger_id

Конструктор (CRUD)

Команда	Описание	Параметры
stands_list/save/load/delete	Стенды	name, data
diagrams_list/save/load/delete	Схемы	name, data
tests_list/save/load/delete	Тесты	name, data
checks_list/save/load/delete	Проверки	name, data
groups_list/save/load/delete	Группы	name, data

Управление тестами

Команда	Описание	Параметры
test_init	Инициализация теста	groupName, checkIndex, testIndex, mode
test_start	Запуск теста	-
test_pause	Пауза	-

Команда	Описание	Параметры
test_resume	Продолжить	-
test_step	Шаг	-
test_stop	Остановить	-
test_status	Получить статус	-
test_result	Получить результат	-
test_save_standard	Сохранить как эталон	-
test_compare_standard	Сравнить с эталоном	tolerance
test_sessions_list	Список сессий	-
test_session_load	Загрузить сессию	sessionId
Результаты		
Команда	Описание	Параметры
result_list	Список результатов	-
result_load	Загрузить результат	name
result_delete	Удалить результат	name
result_compare	Сравнить результаты	name1, name2, tolerance

5. Форматы данных

Тест (tests)

javascript

```
{
  "name": "Тест_открытия_двери",
```

```
"description": "Описание теста",  
"test": [  
  [0, "green", 3],  // [время_мс, команда, значение]  
  [500, "lock", 3],  
  [1000, "red", 2]  
]  
}
```

Проверка (checks)

```
javascript  
{  
  "name": "Проверка_двери",  
  "description": "Описание проверки",  
  "initial": [  
    ["lock", 0],    // [команда, значение]  
    ["green", 0]  
  ],  
  "triggers": [  
    ["door", 32],   // [триггер, состояние]  
    ["card_in", 8]  
  ],  
  "tests": ["Тест_открытия_двери"], // имена тестов  
  "_currentResults": {              // автоматически  
    "Тест_открытия_двери": [[0, "green", 3], ...]  
  },  
  "_standards": {                   // автоматически  
    "Тест_открытия_двери": [[0, "green", 3], ...]  
  }  
}
```

Группа (groups)

```
javascript  
{  
  "name": "Группа_тестов_двери",
```

```
"description": "Описание группы",  
"stand": "Стенд_1",  
"checks": ["Проверка_двери"]    // имена проверок  
}
```

Стенд (stands)

```
javascript  
{  
  "name": "Стенд_1",  
  "description": "Описание стенда",  
  "controllers": ["37_38650"],    // ID контроллеров  
  "diagram": "Схема_стандартная" // имя схемы  
}
```

Схема (diagrams)

```
javascript  
{  
  "name": "Схема_стандартная",  
  "description": "Описание схемы",  
  "url": "/diagrams/files/scheme_1234567890.pdf"  
}
```

Результат (results)

```
javascript  
{  
  "name": "Группа_1_Проверка_двери_Тест_открытия_2026-07-16T13-46-32-304Z",  
  "group": "Группа_1",  
  "check": "Проверка_двери",  
  "test": "Тест_открытия",  
  "timestamp": "2026-07-16T13-46-32-304Z",  
  "result": [  
    [0, "green", 3],  
    [500, "lock", 3]  
  ],  
  "totalSteps": 2,  
}
```

```
"controller": "37_38650",  
"sessionId": "..."  
}
```

Сессия теста (test_sessions)

```
javascript  
{  
  "sessionId": "1234567890_abc123",  
  "group": "Группа_1",  
  "check": "Проверка_двери",  
  "test": "Тест_открытия",  
  "status": "running",    // running | paused | initialized  
  "step": 0,  
  "total": 5,  
  "result": [...],  
  "controller": "37_38650",  
  "mode": "auto"  
}
```

6. Структура классов

Converter

```
javascript  
class Converter {  
  // Управление конвертером Z397IP  
  // Очереди команд: q_fast, q_main  
  // Поиск контроллеров в сети RS485  
}
```

Controller

```
javascript  
class Controller {  
  // Управление контроллером Z-5R Net  
  // Триггеры, паттерны, команды  
}
```


TestRunner

javascript

```
class TestRunner {  
    // Движок выполнения тестов  
    // init(group, checkIndex, testIndex, mode)  
    // start(), pause(), resume(), step(), stop()  
    // saveAsStandard(), compareWithStandard()  
    // Управление сессиями  
}
```

fileOperations

javascript

```
function fileOperations() {  
    // getList(folder) - список файлов  
    // save(folder, name, data) - сохранение  
    // load(folder, name) - загрузка  
    // delete(folder, name) - удаление  
    // checkExists(folder, name) - проверка существования  
    // findReferences(folder, name, searchFolders) - поиск ссылок  
}
```

7. Команды управления контроллерами (Modbus)

Сервер общается с контроллерами через Modbus RTU поверх TCP.

Базовый протокол:

- Команды формируются в байтовые массивы
- CRC16 в конце пакета
- Ответ содержит статус выполнения

Основные команды:

- Чтение состояния (0x04)
- Запись команды (0x06)
- Чтение информации (0x01)

Карта команд:

Команда	Бит	Описание
LED	0x01	Управление светодиодом
ZUMM	0x02	Зуммер
SOUND	0x04	Звук
LOCK	0x08	Замок

8. Запуск и установка

Установка

bash

Клонирование или распаковка проекта

cd project

npm install

Запуск

bash

npm start

или

node index.js

Доступ к страницам

text

http://localhost:8080 # Администрирование

http://localhost:8080/runner.html # Исполнение

9. Конфигурация

Параметры в index.js

javascript

const port = 25000; // TCP сервер

const port_client = 1000; // TCP клиент

const port_http = 8080; // HTTP сервер

const port_ws = 9090; // WebSocket сервер

```
const port_udp = 9000;    // UDP сервер (поиск)
const api_key = '1356_api'; // API ключ
const folder = "site";    // Папка со статикой
```

Переменные окружения

```
bash
```

```
# Порт HTTP сервера
```

```
PORT_HTTP=8080
```

```
# Порт WebSocket
```

```
PORT_WS=9090
```

```
# API ключ
```

```
API_KEY=1356_api
```

10. Отладка

Логирование

Сервер пишет логи в консоль:

```
text
```

```
=====
```

```
 СЕРВЕР УПРАВЛЕНИЯ СКУД v3.0.2
```

```
=====
```

```
 HTTP Server: http://192.168.1.61:8080
```

```
 WS Server: ws://192.168.1.61:9090
```

```
 Создана папка: stands
```

```
 CONNECTED: 0 192.168.1.89:1000
```

```
 Новый контроллер: 37_38650
```

```
 Новый пользователь: 0
```

Клиентский лог

В браузере открыть **F12** → **Console** для просмотра сообщений.

Очистка логов сервера

В index.js закомментированы строки:

```
javascript
```

```
// console.log('📄 Получено:', jsonObj.api);  
  
// console.log('✅ Ответ отправлен клиенту:', id);  
  
// console.log('📄 Результат сохранен: ${resultName}');
```

11. Расширение проекта

Добавление новой команды:

1. Добавить в CMD_LIST на клиенте
2. Добавить в cmdMap в _executeCommand() на сервере
3. Добавить обработчик в func_api если нужна логика на сервере

Добавление нового триггера:

1. Добавить в TRIGGER_LIST на клиенте
2. Добавить в func_api.types на сервере
3. Добавить в TRIGGER_LIST в app.js

Добавление нового типа объекта в конструктор:

1. Добавить вкладку в index.html
 2. Добавить редактор в renderEditor()
 3. Добавить обработчики в initEditorHandlers()
 4. Добавить API команды в out_api
-

12. Структура клиентского кода

index.html (администрирование)

- Весь код в одном файле
- CMD_LIST — доступные команды
- TRIGGER_LIST — доступные триггеры
- state — глобальное состояние
- handleApiResponse() — обработка ответов от сервера
- renderEditor() — отрисовка редакторов конструктора

runner.html (исполнение)

- Весь код в одном файле
- handleTestEvent() — обработка событий теста
- updateRunnerChecks() — обновление списка проверок
- updateRunnerTests() — обновление списка тестов

- `runTest()` — запуск теста